

1 / 2 · 문서에서 검색·답변까지

Reader, Splitter, Index, Retriever, QueryEngine의 입력과 반환값을 연결한다.

실행: documents → nodes/index → retrieve → 답변 | 역추적: 답변 ← 질문 + 검색 문맥

[기출 복기 + 범위 핵심] 로드·청킹·인덱스·실행

```
documents = SimpleDirectoryReader("data").load_data()
index = VectorStoreIndex.from_documents(documents)
# 청킹 규칙을 직접 지정하는 대안
parser = SentenceSplitter(chunk_size=1024, chunk_overlap=200)
nodes = parser.get_nodes_from_documents(documents)
# 또는 from_documents(..., transformations=[text_splitter])
retriever = index.as_retriever()
ret_passages = retriever.retrieve(question)
query_engine = index.as_query_engine()
response = query_engine.query(question)
```

인자 출처 "data": 문제에서 주어진 문서 폴더. documents: Reader 반환 목록. chunk_size·chunk_overlap: 청킹 설정, 숫자는 문제 조건에서 읽기. question: 사용자 질의. index: 앞서 생성한 VectorStoreIndex.

실수 방지 retrieve는 관련 Node 목록, query는 검색과 생성이 합쳐진 응답. nodes는 청킹 결과입니다. chunk 크기 단위는 이 splitter의 토큰 기준이며 overlap < chunk_size로 둡니다.

근거: 1. Llama_index.ipynb · 셀 14~28, 50, QueryEngine 관련 셀

[범위 핵심] CustomQueryEngine 연결

```
# StandardQueryEngine.custom_query(self, query_str)
nodes = self.retriever.retrieve(query_str)
response_obj = self.response_synthesizer.synthesize(query_str, nodes)
return response_obj

# OurCustomQueryEngine.custom_query의 직접 생성 경로
nodes = self.retriever.retrieve(query_str)
context_str = "\n\n".join(n.node.get_content() for n in nodes)
response = self.llm.complete(
    self.qa_prompt.format(context_str=context_str, query_str=query_str))
return str(response)
```

인자 출처 self.retriever: index.as_retriever(). response_synthesizer:

get_response_synthesizer(response_mode="compact"). self.qa_prompt: 앞서 만든 PromptTemplate. format의 키는 템플릿의 {context_str}, {query_str}와 일치.

실수 방지 OurCustomQueryEngine 생성 시 retriever=retriever, response_synthesizer=synthesizer, llm=llm, qa_prompt=simple_qa_prompt. 이 구현은 생성자에 synthesizer를 받아도 custom_query에서는 llm.complete를 직접 씁니다.

근거: 1. Llama_index.ipynb · 셀 76~83, StandardQueryEngine / OurCustomQueryEngine

가리고 쓰기: documents를 받는 함수 / 질문을 받는 함수 / 검색 결과의 text 추출 / format의 두 키.

2 / 2 · llm_input, MCP, 평가

검색 결과를 messages로 조립하고 MCP 도구와 평가 입력을 연결한다.

실행: 검색 chunk → prompt_generator → messages → completion | 역추적: 답변 ← LLM ← references

[기출 복기] Reader의 llm_input

```
# Reader.generate_response(question, query_time, top_k_chunks)
llm_input = self.prompt_generator(question, query_time, top_k_chunks)
completion = oai_client.chat.completions.create(
    model="gpt-3.5-turbo", temperature=0, messages=llm_input
).choices[0].message.content

# prompt_generator(self, query, query_time, top_k_chunks) 안
references = "\n".join(f"- {chunk.strip()}" for chunk in top_k_chunks)
user_message = (f"{references}\nQuestion: {query}"
                f"\nQuery Time: {query_time}")
llm_input = [
    {"role": "system", "content": self.system_prompt},
    {"role": "user", "content": user_message},
]
return llm_input
```

인자 출처 question/query/query_time: 함수 입력. top_k_chunks: retrieval의 검색 결과 문자열 목록. self.system_prompt: Reader.__init__. oai_client: from openai import OpenAI로 만든 client. model 문자열은 문제에서 준 설정 사용.

실수 방지 messages는 문자열 하나가 아닌 role/content 딕셔너리 목록. LlamaIndex OpenAI.complete와 OpenAI SDK chat.completions.create를 섞지 않습니다. 위 prompt 조립은 원본의 핵심만 압축한 예입니다.

근거: 4_RAG_framework_evaluation_with_MCP.ipynb · 셀 40 Reader; Task_1/Task_2의 동일 흐름

[범위 핵심] MCP 도구와 평가

```
mcp_client = BasicMCPClient(external_mcp_server)
mcp_tool = McpToolSpec(client=mcp_client)
tools = await mcp_tool.to_tool_list_async()
# init_agent 안: tools와 llm은 앞에서 준비
self.agent = FunctionAgent(
    name="Agent", tools=tools, llm=self.llm,
    system_prompt=SYSTEM_PROMPT)
self.agent_context = Context(self.agent)

# 평가 함수 입력은 question, ground_truth, prediction
evaluation_result = generate_answer(
    user_prompt=context_template, system_prompt=INSTRUCTIONS)
eval_res = parse_response(evaluation_result)
```

인자 출처 external_mcp_server: 주어진 서버 주소. mcp_tool: 바로 위 client를 감싼 도구 명세.

SYSTEM_PROMPT·INSTRUCTIONS: 앞선 셀 정의. context_template: 질문·정답·모델 답변을 삽입한 문자열.

실수 방지 비동기 결과는 await로 받습니다. 평가 정답 ground_truth를 실제 답변 생성 프롬프트에 넣지 않습니다. API 키·서버 주소 암기보다 인자 연결과 반환값을 우선합니다.

근거: 4_RAG_framework_evaluation_with_MCP.ipynb · 셀 20,43,47

가리고 쓰기: llm_input의 두 role / messages 인자 / .choices[0].message.content / MCP tools 획득.